

MiniANN Project Documentation

PROJECT OVERVIEW

MiniANN is a tiny C++ neural network library for training simple networks (XOR, AND, OR) with a Qt Widgets GUI. The project has three parts: Core Math, Experiment (training pipeline), and GUI.

HOW TO RUN THE PROJECT

Method 1: build.bat (recommended)

```
\\ .\\build.bat
```

Compiles 11 targets (xor_demo.exe through gui_qt.exe). After building:

```
\\ .\\gui_qt.exe
```

Method 2: CMake + Ninja

```
\\ cmake -B build -G Ninja
```

```
\\ cmake --build build --target gui_qt
```

```
\\ .\\build\\gui_qt.exe
```

CODE OVERVIEW FILE BY FILE

`demos/gui_qt.cpp` The Qt GUI window — left panel (datasets, config, presets), right panel (Train/Validate/Test/Network tabs), bottom bar (SAVE/LOAD/START + Speed Slider + [Step 1] button). **New in this session:** canvas text overlap fix, speed slider, step button, left bar formatting.

`include/miniann/optimizer.hpp` Optimizer strategy pattern — `IOptimizer` base class with virtual `step()` and `name()`; SGD, Adam, Momentum inherit and implement. **New in this session:** getter methods for all three, `State` structs with `state()/restore()`, `name()` overrides returning “sgd”, “adam”, “momentum”.

`include/miniann/loss.hpp` Loss function system — `ILoss` base class with virtual `compute()`, `gradient()`, `name()`; `MSELoss`, `BCELoss`, `CCELoss` inherit. **New in this session:** virtual `std::string name() const = 0` added to interface, `name()` overrides in all three classes.

`src/experiment.cpp` Training pipeline — `ExperimentController::run()` in a `std::thread`, writes live metrics to shared `g_bridge` (mutex-protected). **Fixed in this session:** stratified train/val/test splits, per-feature max-abs normalization, He init for ReLU family, preview-call dedup.

`src/experiment.hpp` Declares `ExperimentConfig`, `ExperimentResult`, streaming sink types.

`build.bat` Build script — 11/11 step numbering, [10/11] `test_correctness.exe`, [11/11] `gui_qt.exe`.
New: UCRT g++ auto-selection, `deploy_qt.ps1`.

`deploy_qt.ps1` PowerShell script — makes `gui_qt.exe` work by double-click even with other Qt on PATH. Run once after building.

`tests/test_choices.cpp` Minor updates to choices test.

OOP CONCEPTS IN THIS PROJECT

Each subsection below has a short **heads-up** summary before the details.

Abstraction (Interfaces)

Heads-up: Abstraction means defining a common interface (function signatures) that other classes must follow. The code programs to the interface, not the concrete implementation.

Where: `include/miniann/optimizer.hpp` :10-15 — `IOptimizer` class with pure virtual `step()` and `name()`. Similarly `include/miniann/loss.hpp` :10-15 — `ILoss` class with virtual `compute()`, `gradient()`, `name()`.

Details: SGD, Adam, Momentum all inherit from `IOptimizer` and implement the two pure virtual functions. The GUI holds a `std::unique_ptr<IOptimizer>` — it points to a concrete optimizer but the GUI only knows the interface type. When you call `optimizer->step(net, batchSize)`, the correct implementation fires based on the actual object type (polymorphism). Same pattern for loss functions — the GUI doesn't need to know whether MSE, BCE, or CCE is active; it just calls the interface methods.

Polymorphism (Virtual Functions)

Heads-up: Polymorphism means same function name, different behavior depending on the actual object type. C++ decides at runtime which version to call.

Where: `demos/gui_qt.cpp` — `g_bridge.optimizer->step(net, batchSize)` calls the correct optimizer's `step()`. Similarly `optimizer->name()` returns “sgd”, “adam”, or “momentum” depending on which optimizer is active.

Details: `g_bridge.optimizer` is a `std::unique_ptr<IOptimizer>` — it could point to an SGD, Adam, or Momentum object. When `step()` is called, C++ looks at the actual object type and runs that optimizer's `step()`. Liken: same `play()` button on different devices — CD player spins a disc, Spotify streams, radio turns on. Here, call `optimizer->step()`, but behavior depends on which optimizer is running.

Encapsulation (Data Hiding)

Heads-up: Encapsulation means keeping data private inside a class; expose only what's needed through methods. Prevents outside code from messing with internal state.

Where: `include/miniann/optimizer.hpp` private members + getters (`SGD::lr_` → `lr()`, `Adam::m_w_/m_b_/t_` → `State` struct with `state()/restore()`). `g_bridge.mtx` protects shared state between GUI and training threads. `NetWidget::m_tag`, `m_net`, `m_probe` private with access through `setNet()`.

Details: The GUI never touches `SGD::lr_` directly — it calls `optimizer->lr()` (the getter). The training thread writes to `g_bridge` under `std::lock_guard`/`std::mutex` — the GUI reads `g_bridge` values safely. `NetWidget::m_hit` (hit-test rectangles for tooltips) is private; `mouseMoveEvent()` checks them but doesn't modify them. Liken: car — you turn steering wheel and press gas (exposed interfaces), but can't reach in and mess with engine timing belt (hidden inside). Prevents accidental breaking.

Factory Pattern

Heads-up: Factory pattern creates objects without specifying the exact class type. Ask for “an optimizer named adam” and the factory gives you one.

Where: `include/miniann/optimizer.hpp` :92-97 — `OptimizerFactory::create()` returns `std::unique_ptr<IOptimizer>` based on name string.

Details: 1. GUI combobox selects optimizer name: “sgd”, “adam”, or “momentum”. 2. `currentConfig()` builds `OptimizerConfig` from widget values. 3. `OptimizerFactory::create(name, config)` returns the correct concrete optimizer. 4. GUI stores it in `g_bridge.optimizer` (type `IOptimizer`). 5. Training calls `optimizer->step(net, batchSize)` — polymorphic call. Liken: vending machine — press button for “Adam”, machine gives you Adam optimizer object. You don't need to know how to construct an Adam — the factory handles that. GUI just says “give me optimizer name X” and factory delivers it.

Composition (Over Inheritance)

Heads-up: Composition builds complex things by composing simpler objects, rather than one giant class inheriting from many base classes.

Where: `NetWidget` has `QPainter`, `QFontMetrics`, hit rectangles, tooltips (`demos/gui_qt.cpp`). `ExperimentController` has `g_bridge`, `g_trainThread`, `currentConfig()` (`demos/gui_qt.cpp`). `NeuralNetwork` has `std::vector<Layer>`, each `Layer` has `std::vector<Neuron>` (`include/miniann/network.hpp`).

Details: `NetWidget` doesn't inherit from giant `NetworkDrawer`. Instead, it has a `QPainter`, has a font metrics object, has a hit-test rectangle vector. It delegates drawing to `QPainter`, tooltip logic to `QToolTip`. `ExperimentController` composes a thread, shared bridge, and config values. `NeuralNetwork` has-a vector of Layers, each Layer has-a vector of Neurons. Not “is-a” relationships. Liken: LEGO bricks — each brick has one purpose, snap together to make bigger structure.

Threading + Mutex (Concurrency)

Heads-up: Threading runs code in separate thread so main program doesn't freeze. Mutex (mutual exclusion) prevents two threads from accessing same data at same time (would crash).

Where: `src/experiment.cpp` — `g_bridge.mtx`, `g_trainThread = std::make_unique<std::thread>(...)`. GUI reads `g_bridge` under mutex, training thread writes `g_bridge` under `std::lock_guard<std::mutex>`.

Details: `onTrainClicked()` launches `std::thread` running `ExperimentController::run()`. Training thread writes metrics to `g_bridge` — always under `std::lock_guard<std::mutex>`. `refreshLiveUi()` (Qt timer, 100ms) reads `g_bridge` — also under `std::lock_guard<std::mutex>`. Ensures: while training thread writes “epoch 42, loss 0.123”, GUI isn't in middle of reading epoch 42's data → no garbled values, no crashes. Liken: bathroom with “Occupied” sign — when someone's inside (training thread), sign is up (mutex locked). GUI (waiting outside) can't enter until person inside finishes and leaves (releases mutex). Prevents two people in bathroom at same time.

EXECUTION TRACE WHAT HAPPENS WHEN YOU RUN IT

Step 0: Application starts → `gui_qt.exe` launches, `QApplication` created, main window shown, left panel config visible, right panel tabs (Train/Validate/Test/Network), bottom: SAVE/LOAD/START + Speed Slider + [Step 1].

Step 1: User configures and clicks [START TRAINING] → User sets XOR, 2 hidden layers, Tanh, Adam, 1500 epochs, lr 0.01 → clicks START → `onTrainClicked()` fires.

Step 2: Training runs in background → `onTrainClicked()` launches `std::thread` running `ExperimentController::run(cfg, &g_bridge, &g_bridge.stopRequested)`. Thread iterates epochs: forward pass → compute loss (MSE/BCE/CCE) → compute gradients → `optimizer->step(net, batchSize)` → update weights → evaluate accuracy → write live data to `g_bridge` UNDER mutex → check `stopRequested`.

Step 3: GUI reads live data → Every 100ms Qt timer fires `refreshLiveUi()`. Reads `g_bridge` UNDER mutex → updates labels (epoch, train loss, train acc, val acc, test acc) → updates plot only if data changed (`m_cPlotA/m_cPlotB/m_cPlotV/m_cPlotE` gate prevents flicker on idle).

Step 4: Training completes → `ExperimentController::run()` exits thread → `g_bridge.hasResult=true`, `g_bridge.lastCompleted=true/false`, `g_bridge.lastNet=trained pointer`, full history populated → `std::thread.join()` in `onTrainClicked()` → GUI resumes.

Step 5: GUI updates to show results → `refreshConfigUi()` runs → metric labels set → status bar badge (COMPLETED/STOPPED/ERROR) → `m_netview->setNet(net, probe, tag)` → `tag = “TRAINED — 2 -> 8(tanh)”` (from `describeNet()`) → speed slider respected for render density → [Step 1] button can advance one step.

Step 6: User probes neurons → `mouseMoveEvent()` checks `m_hit` rectangles → if inside → `QToolTip::showText()` with: input neuron → “Input x1 = 0.5”; hidden/output neuron → “L1N2 b=0.35 out=0.82=[0.12, -0.45, 0.91]”.

Step 7: User steps forward incrementally → [Step 1] button launches single-epoch training thread (same code path as START, but only one run completes).

Step 8: Save/Load model → SAVE MODEL → serialize `NeuralNetwork` to `.model` file → LOAD MODEL → browse to `.model` → network appears in GUI.

Step 9: Switch tabs → Train tab (metrics plot), Validate tab (validation metrics), Test tab (test metrics + confusion matrix), Network tab (network diagram with live neuron firing).

QUICK REFERENCE WHERE EVERYTHING IS

Concern	File	Line/Section
OOP Abstraction	<code>include/miniann/optimizer.hpp</code>	:10-15 (<code>IOptimizer</code>)
OOP Polymorphism	<code>demos/gui_qt.cpp</code>	<code>g_bridge.optimizer->step()</code>
OOP Encapsulation	<code>include/miniann/optimizer.hpp</code>	:19-56 (private + getters)
OOP Factory	<code>include/miniann/optimizer.hpp</code>	:92-97 (<code>OptimizerFactory</code>)
OOP Loss Interface	<code>include/miniann/loss.hpp</code>	:10-15 (<code>ILoss</code>)
Training Thread	<code>src/experiment.cpp</code>	:2188-2236 (<code>g_trainThread</code>)
Shared State (<code>g_bridge</code>)	<code>src/experiment.cpp</code> + global	Mutex-protected
<code>NetWidget</code> Canvas	<code>demos/gui_qt.cpp</code>	:562-687 (<code>paintEvent</code>)
Speed Slider	<code>demos/gui_qt.cpp</code>	:1607-1658 (<code>m_speedSlider</code>)
Canvas Text Fix	<code>demos/gui_qt.cpp</code>	:568, :684 (inset 22→36px)
Build Script	<code>build.bat</code>	:36-76 (11/11 numbering)
Qt Deployment	<code>deploy_qt.ps1</code>	Run once after building

FAQ COMMON QUESTIONS

Q: Where is OOP used in this project? A: 1. `include/miniann/optimizer.hpp` — `IOptimizer` base class with virtual `step()` and `name()`; SGD, Adam, Momentum inherit and implement. 2. `include/miniann/loss.hpp` — `ILoss` base class with virtual `compute()`, `gradient()`, `name()`; `MSELoss`, `BCELoss`, `CCELoss` inherit. 3. `demos/gui_qt.cpp` — `NetWidget` composes `QPainter`, `QFontMetrics`, hit rectangles, tooltips (composition). 4. `src/experiment.cpp` — `g_bridge` mutex-protected shared state read by GUI, written by thread; `g_trainThread` for background training. 5. `include/miniann/optimizer.hpp` — `OptimizerFactory::create()` returns `std::unique_ptr<IOptimizer>` based on name. 6. `demos/gui_qt.cpp` — `onTrainClicked()` launches `std::thread`; `currentConfig()` builds `OptimizerConfig`; factory creates optimizer.

Q: How does training actually happen when I run `gui_qt.exe`? A: 1. Launch app → main window appears. 2. Configure settings (dataset, layers, activation, opti-

mizer, epochs) and click START TRAINING. 3. onTrainClicked() launches background std::thread running ExperimentController::run(). 4. Thread trains for specified epochs, writing live metrics (loss, accuracy) to shared g_bridge struct (protected by mutex). 5. Every 100ms, Qt timer fires refreshLiveUi(), reads g_bridge under mutex, updates labels and plot. 6. When training finishes, thread joins back, GUI displays final metrics, shows network diagram, enables Save/Load model.

Q: What does the Speed Slider do? A: Controls epochs rendered per GUI update tick. Value 1 = render every epoch (fastest visible). Value 20 = render 1 epoch every 20 actual epochs (slow-mo, helpful for watching learning on XOR/AND/OR). [Step 1] button advances training by one step, respecting speed setting.

Q: Why was the TRAINED tag cutting through neurons? A: Tag drawn at area.top() + 2 inside frame with 22px top inset. First neuron row at area.top() + 18, leaving only 16px for tag text plus DPI scaling. Fix: increased top inset from 22px to 36px (6px above tag, 20px between tag and first neuron row), moved tag y-position from area.top() + 2 to area.top() + 6.

Q: Can I add more datasets? A: Yes — AND, OR, XOR, Iris, CSV buttons in left panel. Add more by adding buttons, data generation code, possibly adjust boundary visualization for 2D inputs.

Q: How do I save and load a trained network? A: SAVE MODEL → serialize NeuralNetwork to .model file (layer sizes, weights, activations). LOAD MODEL → browse to .model → network appears in GUI, ready for Network tab or further training.

Q: What if I want a different optimizer or loss? A: Use dropdown comboboxes in left config panel. Optimizer: SGD, Adam, Momentum. The OptimizerFactory creates the right object by name. ILoss interface ensures all losses have same API (compute(), gradient(), name()).

Q: Where can I learn more about the math inside? A: include/miniann/: network.hpp (NeuralNetwork, Layer, Neuron structs, Vector type), optimizer.hpp (how step() updates weights), loss.hpp (how compute() and gradient() calculate error), experiment.hpp (training pipeline and result types), src/*.cpp (actual implementations).